

AIKB 系统架构与服务边界

- 文档编号：AIKB-ARCH-001
- 适用系统：企业知识库 RAG 平台
- 文档状态：与当前仓库实现一致

1. 架构目标

AIKB 把企业知识库 RAG 作为主业务，把求职 Copilot 作为共享 AI 基础设施的扩展案例。系统采用 Spring Boot 业务后端与 FastAPI AI 服务分层的结构，避免用户权限、业务状态和模型调用全部耦合在一个进程中。前端由 Spring Boot 静态资源提供，浏览器只访问业务后端，不直接信任用户提交的身份参数。

2. Spring Boot 业务控制面

Spring Boot 负责注册登录、JWT 校验、知识库管理、知识库访问权限、文档状态、聊天会话与消息记录。文档上传时，它还负责文件类型检查、SHA-256 重复检测、Redis 防重复锁和 AI 调用限流。Spring Boot 不解析 PDF，也不直接生成向量；这些 AI 数据处理职责交给 FastAPI。

文档业务状态保存在 MySQL。上传开始时先写入 PROCESSING，FastAPI 入库成功后更新为 AVAILABLE；调用失败时更新为 FAILED 并保存错误原因。这样前端能区分正在处理、可用和失败，而不是只依赖一次 HTTP 请求是否返回 200。

3. FastAPI AI 数据面

FastAPI 负责 PDF 文本提取、按页切分 Chunk、调用 text-embedding-v4 生成 1024 维稠密向量、生成稀疏词法向量、写入 Qdrant、执行 Dense/Sparse 检索、RRF 融合、Qwen Rerank、构建引用 Prompt 以及调用大语言模型生成答案。

Spring Boot 通过 Docker 内部地址 `http://api:8000` 调用 FastAPI。FastAPI 内部接口可以使用 `FASTAPI_API_KEY` 做服务间校验；该配置留空时方便本地开发，但面向共享环境部署时应设置非空值。

4. 数据存储职责

MySQL 保存结构化业务事实，包括用户、知识库、文档元数据与状态、聊天会话、聊天消息、AI 调用日志和求职扩展业务记录。Qdrant 保存可检索的 Chunk、Dense/Sparse 向量及 `filename`、`page_number`、`chunk_index`、`document_id`、`file_hash` 等引用载荷。Redis 保存短期且高频变化的状态，目前用于 AI

调用限流计数和上传防重复锁。

MySQL 中的 KnowledgeDocument 与 Qdrant 中的 document_id 形成业务状态到向量数据的关联。

MySQL 不是向量检索引擎，Qdrant 也不承担用户权限和聊天事务。

5. 文档入库时序

1. 浏览器携带 Bearer JWT 向知识库文档接口上传 PDF。
2. Spring Boot 校验用户能访问目标知识库，并检查 PDF、文件 Hash、限流与上传锁。
3. Spring Boot 写入 PROCESSING 文档记录，然后通过 multipart/form-data 调用 FastAPI。
4. FastAPI 提取每页文本，按句子切分并保留页码，同时生成 Dense 与 Sparse 向量。
5. FastAPI 把 Chunk 写入 rag_chunks_hybrid_v1，并返回 document_id 与 chunk_count。
6. Spring Boot 将文档更新为 AVAILABLE；异常时更新为 FAILED，最终释放 Redis 上传锁。

6. 问答时序

用户先创建绑定 knowledgeBaseId 和 documentId 的聊天会话，再向会话 ask 接口提问。Spring Boot 验证会话、知识库和文档访问权，随后调用 FastAPI 的 /rag/chat/rerank。FastAPI 完成 Hybrid RRF 候选召回、Rerank、阈值拒答与带引用生成。Spring Boot 保存用户消息和助手回答，并把来源信息返回前端。

当前知识库访问规则是：创建者可以访问自己创建的知识库，同部门用户也可以访问该部门知识库；其他用户被拒绝。该规则由业务后端执行，不能只依靠前端隐藏按钮。